# Socket.IO Singleton Pattern — Implementation Guide

Building one shared socket connection in a Next.js app: the pattern, the final code, and how to connect/disconnect it correctly around auth.

---

## Why a Singleton?

In a messaging system, every component doesn't need its own Socket.IO connection. If multiple components each create their own socket instance, they can end up joining rooms independently — a message broadcast to "your" room might land on a socket instance no component is listening to. It also wastes server resources: N components means N open connections in the server's connection pool for a single logical user.

The fix: one file whose only job is to create the socket, configure it, and always return the same instance.

## What This File's Job Is

- Create ONE socket instance shared across the whole app
- Guard against server-side execution (Next.js SSR has no **window**)
- Leave events, state updates, and business logic to other files

## Final lib/socket.js

```
"use client";

import { io } from "socket.io-client";

let socketInstance = null;

export const getSocket = () => {
  if (typeof window === "undefined") {
    return null;
  }

  if (!socketInstance) {
    const serverUrl =
      process.env.NEXT_PUBLIC_SERVER_BASE_URL || window.location.origin;

    socketInstance = io(serverUrl, {
      withCredentials: true,
      autoConnect: false,
      transports: ["websocket"],
    });
  }

  return socketInstance;
};
```

```
export const resetSocket = () => {
  if (socketInstance) {
    socketInstance.removeAllListeners();
    socketInstance.disconnect();
  }

  socketInstance = null;
};

export default getSocket;
```

Note: **transports: ["websocket"]** skips Socket.IO's HTTP long-polling fallback. Faster, but if users sit behind proxies/firewalls that block WebSocket upgrades, there's no fallback — only keep this if you've verified it in your deployment environment.

## Connect After Login

Don't call **getSocket()** on component mount. Only connect once the user is confirmed authenticated — usually from a top-level provider tied to your auth state.

```
// SocketProvider.jsx
"use client";

import { useEffect } from "react";
import { getSocket } from "@/lib/socket";
import { useAuth } from "@/context/AuthContext";

export default function SocketProvider({ children }) {
  const { user, isAuthenticated } = useAuth();

  useEffect(() => {
    if (!isAuthenticated || !user) return;

    const socket = getSocket();

    // pass auth token/userId so server can identify + join rooms
    socket.auth = { userId: user.id, token: user.token };
    socket.connect();

    socket.on("connect", () => {
      console.log("Socket connected:", socket.id);
    });

    return () => {
      // cleanup listeners on unmount, but don't disconnect here
      socket.off("connect");
    };
  }, [isAuthenticated, user]);

  return children;
}
```

## Disconnect on Logout

Call **resetSocket()** from the actual logout handler — not from a **useEffect** cleanup. A cleanup fires on every unmount, including normal page navigation, which would kill the socket for no reason. Tying it to the logout action ensures it only fires when the user truly signs out.

```
// logout handler
import { resetSocket } from "@/lib/socket";

const handleLogout = async () => {
  await signOut();   // clears cookies/tokens
  resetSocket();      // kill the socket connection
  router.push("/login");
};
```

### Bonus: Handling Token Refresh

If auth tokens refresh silently (common with JWT flows), the socket's **auth** payload can go stale. Reconnect with a fresh token on error:

```
socket.on("connect_error", (err) => {
  if (err.message === "invalid_token") {
    socket.auth = { userId: user.id, token: newToken };
    socket.connect();
  }
});
```

## Quick Recap

- **One instance** — lib/socket.js only creates, configures, and returns the socket.
- **SSR-safe** — guarded with typeof window === "undefined".
- **Connect** — triggered after auth is confirmed, not on component mount.
- **Disconnect** — triggered from the logout action, not a useEffect cleanup.
- **Listeners** — cleared with removeAllListeners() before disconnecting, to avoid stale bindings.